

```

//PEDRO
#include <iostream>
#include <string>
#include <iomanip>
#include <cstring>
using namespace std;

/* ----- */
// Exercícios de Vetores Básico:
/*
int main() {
    // Exercício 5
    int vetor[8];
    int x, y;

    cout << "Exercício 5:\n";
    cout << "Digite 8 valores para o vetor:\n";
    for (int i = 0; i < 8; i++) {
        cin >> vetor[i];
    }
    cout << "Digite as posições X e Y: ";
    cin >> x >> y;
    cout << "Soma dos valores nas posições " << x << " e " << y << ": " << vetor[x] + vetor[y] <<
endl;

    // Exercício 6
    int vetor2[10];
    int pares = 0;

    cout << "\nExercício 6:\n";
    cout << "Digite 10 valores para o vetor:\n";
    for (int i = 0; i < 10; i++) {
        cin >> vetor2[i];
        if (vetor2[i] % 2 == 0) {
            pares++;
        }
    }
    cout << "Quantidade de valores pares: " << pares << endl;

    // Exercício 7
    int vetor3[10];
    int maior, menor;

    cout << "\nExercício 7:\n";
    cout << "Digite 10 valores para o vetor:\n";
    for (int i = 0; i < 10; i++) {

```

```

    cin >> vetor3[i];
    if (i == 0) {
        maior = menor = vetor3[i];
    } else {
        if (vetor3[i] > maior) {
            maior = vetor3[i];
        }
        if (vetor3[i] < menor) {
            menor = vetor3[i];
        }
    }
}
cout << "Maior valor: " << maior << endl;
cout << "Menor valor: " << menor << endl;

```

// Exercício 8

```

float vetor4[10];
int negativos = 0;
float somaPositivos = 0;

cout << "\nExercício 8:\n";
cout << "Digite 10 valores reais para o vetor:\n";
for (int i = 0; i < 10; i++) {
    cin >> vetor4[i];
    if (vetor4[i] < 0) {
        negativos++;
    } else {
        somaPositivos += vetor4[i];
    }
}
cout << "Quantidade de números negativos: " << negativos << endl;
cout << "Soma dos números positivos: " << somaPositivos << endl;

```

// Exercício 9

```

int vetorA[10], vetorB[10], vetorC[10];

cout << "\nExercício 9:\n";
cout << "Digite 10 valores inteiros para o vetor A:\n";
for (int i = 0; i < 10; i++) {
    cin >> vetorA[i];
}
cout << "Digite 10 valores inteiros para o vetor B:\n";
for (int i = 0; i < 10; i++) {
    cin >> vetorB[i];
    vetorC[i] = vetorA[i] - vetorB[i];
}

```

```
cout << "Vetor C:\n";
for (int i = 0; i < 10; i++) {
    cout << vetorC[i] << " ";
}
cout << endl;
```

```
// Exercício 10
int vetor5[100];
int indice = 0;
```

```
cout << "\nExercício 10:\n";
for (int i = 1; i <= 100; i++) {
    if (i % 7 != 0) {
        vetor5[indice++] = i;
    }
}
cout << "Vetor:\n";
for (int i = 0; i < 100; i++) {
    cout << vetor5[i] << " ";
}
cout << endl;
```

```
// Exercício 11
float vetor6[20], vetor7[20];
int tamanho;
```

```
cout << "\nExercício 11:\n";
cout << "Digite a quantidade de números (máximo 20): ";
cin >> tamanho;
cout << "Digite " << tamanho << " números reais:\n";
for (int i = 0; i < tamanho; i++) {
    cin >> vetor6[i];
    vetor7[i] = vetor6[i] * vetor6[i];
}
cout << "Vetor original:\n";
for (int i = 0; i < tamanho; i++) {
    cout << fixed << setprecision(2) << vetor6[i] << " ";
}
cout << "\nVetor com quadrados:\n";
for (int i = 0; i < tamanho; i++) {
    cout << fixed << setprecision(2) << vetor7[i] << " ";
}
cout << endl;
```

```
// Exercício 12
int vetor8[10];
```

```

bool repetidos[10] = {false};

cout << "\nExercício 12:\n";
cout << "Digite 10 valores para o vetor:\n";
for (int i = 0; i < 10; i++) {
    cin >> vetor8[i];
    if (repetidos[vetor8[i] - 1]) {
        cout << "Valor repetido: " << vetor8[i] << endl;
    } else {
        repetidos[vetor8[i] - 1] = true;
    }
}

// Exercício 13
int vetor9[10];
bool usado[10] = {false};

cout << "\nExercício 13:\n";
cout << "Digite 10 valores diferentes para o vetor:\n";
for (int i = 0; i < 10; i++) {
    cin >> vetor9[i];
    bool encontrado = false;
    for (int j = 0; j < i; j++) {
        if (vetor9[i] == vetor9[j]) {
            encontrado = true;
            break;
        }
    }
    if (encontrado) {
        cout << "Valor repetido, digite outro número: ";
        i--;
    } else {
        usado[vetor9[i] - 1] = true;
    }
}
cout << "Vetor final:\n";
for (int i = 0; i < 10; i++) {
    if (usado[i]) {
        cout << i + 1 << " ";
    }
}
cout << endl;

return 0;
}
*/

```

```

/* ----- */

/* ----- */
/*
// Cálculo de Notas de Alunos com Matrizes e Strings:
#define N_ALUNOS 100 // Define o número máximo de alunos
#define CARGA_HORARIA 60 // Carga horária total do curso
int main() {
    string nomes[N_ALUNOS]; // Array para armazenar os nomes dos alunos
    float notas[N_ALUNOS][9]; // Matriz para armazenar as notas
    string situacao[N_ALUNOS]; // Array para armazenar a situação dos alunos
    int aprovados = 0, reprovados = 0; // Contadores de aprovados e reprovados
    int qntd = 0;

    cout << "Informe a quantidade de alunos na turma: ";
    cin >> qntd;
    cin.ignore();

    // Coletando dados dos alunos
    for (int i = 0; i < qntd; i++) {
        cout << "Digite o nome do aluno " << (i + 1) << ": ";
        getline(cin, nomes[i]);

        cout << "Nota do Trabalho 1 (0-4): ";
        cin >> notas[i][0]; // Coluna 0
        cout << "Nota da Prova 1 (0-6): ";
        cin >> notas[i][1]; // Coluna 1
        cout << "Nota do Trabalho 2 (0-4): ";
        cin >> notas[i][3]; // Coluna 3
        cout << "Nota da Prova 2 (0-6): ";
        cin >> notas[i][4]; // Coluna 4
        cout << "Total de faltas: ";
        cin >> notas[i][7]; // Armazena o total de faltas
        cin.ignore();

        // Cálculo da soma de trabalhos e provas
        notas[i][2] = notas[i][0] + notas[i][1]; // Coluna 2
        notas[i][5] = notas[i][3] + notas[i][4]; // Coluna 5

        // Cálculo da nota final (média das somas)
        notas[i][6] = (notas[i][2] + notas[i][5]) / 2; // Coluna 6

        notas[i][8] = (CARGA_HORARIA - notas[i][6]) / CARGA_HORARIA * 100; //
        Percentual de presença na Coluna 7

        // Verifica a situação com base na nota final e presença

```

```

        if (notas[i][6] >= 6 && notas[i][8] >= 75) {
            situacao[i] = "Aprovado";
            aprovados++;
        } else {
            situacao[i] = "Reprovado";
            reprovados++;
        }
    }

    // Exibindo os resultados
    //setw para espaçar corretamente os resultados, em formato de planilha
    cout << "\nResultados:\n";
    cout << setw(20) << "Nome" << setw(10) << "T1" << setw(10) << "P1"
        << setw(10) << "T1+P1" << setw(10) << "T2" << setw(10) << "P2"
        << setw(10) << "T2+P2" << setw(10) << "NF" << setw(10) << "TF" << setw(10) << "%
Presenca" << setw(10) << "Situacao" << endl;

    for (int i = 0; i < qntd; i++) {
        cout << setw(20) << nomes[i]
            << setw(10) << notas[i][0]
            << setw(10) << notas[i][1]
            << setw(10) << notas[i][2]
            << setw(10) << notas[i][3]
            << setw(10) << notas[i][4]
            << setw(10) << notas[i][5]
            << setw(10) << fixed << setprecision(2) << notas[i][6]
            << setw(10) << notas[i][7]
            << setw(10) << notas[i][8]
            << setw(10) << situacao[i] << endl;
    }

    cout << "\nAprovados: " << aprovados << endl;
    cout << "Reprovados: " << reprovados << endl;

    return 0;
}
*/
/* -----*/

/* -----*/
// Strings e Manipulações

// Exemplo de manipulações básicas com as funções
/*
int main() {
    // Exemplo 1: strlen()
    char str1[] = "Hello, World!";

```

```

int comprimento = strlen(str1);
cout << "Comprimento da string \'" << str1 << "\": " << comprimento << endl;

// Exemplo 2: strcpy()
char str2[50]; // Cria um espaço suficiente para a nova string
strcpy(str2, str1); // Copia str1 para str2
cout << "String copiada: " << str2 << endl;

// Exemplo 3: strcat()
char str3[100] = "Welcome to "; // Cria uma string inicial
strcat(str3, str1); // Concatena str1 a str3
cout << "String concatenada: " << str3 << endl;

// Exemplo 4: strcmp()
char str4[] = "Hello, World!";
char str5[] = "Hello, World!";
char str6[] = "Goodbye, World!";

int resultado1 = strcmp(str4, str5); // Compara str4 e str5
int resultado2 = strcmp(str4, str6); // Compara str4 e str6

if (resultado1 == 0) {
    cout << "\'" << str4 << "\" é igual a \'" << str5 << "\" << endl;
} else {
    cout << "\'" << str4 << "\" é diferente de \'" << str5 << "\" << endl;
}

if (resultado2 < 0) {
    cout << "\'" << str4 << "\" é menor que \'" << str6 << "\" << endl;
} else if (resultado2 > 0) {
    cout << "\'" << str4 << "\" é maior que \'" << str6 << "\" << endl;
}

return 0;
}
*/

// Código de Cesar
/*
int main() {
    string original, codificada;
    int deslocamento = 3; // Número de posições para deslocar

    cout << "Digite a string: ";
    getline(cin, original);

```

```

// Percorrendo cada caractere da string original
for (int i = 0; i < original.length(); i++) {
    char c = original[i];

    // Verifica se o caractere é uma letra
    if (c >= 'a' && c <= 'z') {
        // Desloca a letra e garante que permaneça dentro do alfabeto
        c = (c - 'a' + deslocamento) % 26 + 'a';
    } else if (c >= 'A' && c <= 'Z') {
        // Desloca a letra maiúscula
        c = (c - 'A' + deslocamento) % 26 + 'A';
    }

    // Adiciona o caractere codificado à nova string
    codificada += c;
}

cout << "String codificada: " << codificada << endl;

return 0;
}
*/

```

```

// Ordenação de Strings
/*
int main() {
    string str1, str2;

    cout << "Digite a primeira string: ";
    getline(cin, str1);

    cout << "Digite a segunda string: ";
    getline(cin, str2);

    // Comparando as duas strings
    if (str1 < str2) {
        cout << "Strings em ordem alfabética:\n";
        cout << str1 << endl;
        cout << str2 << endl;
    } else {
        cout << "Strings em ordem alfabética:\n";
        cout << str2 << endl;
        cout << str1 << endl;
    }
}

```



```

    return 0;
}
*/
/* -----*/

/* -----*/
//Matrizes Diagonais e Calculos
/*
#define N 3 // Dimensão da matriz
int main() {
    int matriz[N][N];

    // Preenchendo a matriz (manualmente)
    cout << "Preencha a matriz " << N << "x" << N << ":\n";
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            cout << "Elemento [" << i << "][" << j << "]: ";
            cin >> matriz[i][j];
        }
    }

    // Exibindo a matriz (percorrer as posições normalmente)
    cout << "\nMatriz:\n";
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            cout << setw(4) << matriz[i][j];
        }
        cout << endl;
    }

    // Cálculo das Somas:

    // Cálculo da soma dos itens acima da diagonal principal
    // Acima da Diagonal Principal: Soma os elementos onde a linha é menor que a coluna
    (i < j).
    int somaAcimaDiagonalPrincipal = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i < j) { // Elementos acima da diagonal principal
                somaAcimaDiagonalPrincipal += matriz[i][j];
            }
        }
    }

    cout << "\nSoma dos itens acima da diagonal principal: " <<
    somaAcimaDiagonalPrincipal << endl;
}

```

```

// Cálculo da soma dos itens abaixo da diagonal principal
Abaixo da Diagonal Principal: Soma os elementos onde a linha é maior que a coluna (i >
j).
int somaAbaixoDiagonalPrincipal = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        if (i > j) { // Elementos abaixo da diagonal principal
            somaAbaixoDiagonalPrincipal += matriz[i][j];
        }
    }
}
cout << "Soma dos itens abaixo da diagonal principal: " << somaAbaixoDiagonalPrincipal
<< endl;

// Cálculo da soma dos itens acima da diagonal secundária
// Acima da Diagonal Secundária: Soma os elementos onde a linha é menor que N - 1 - j
(i < N - 1 - j).
int somaAcimaDiagonalSecundaria = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        if (i < N - 1 - j) { // Elementos acima da diagonal secundária
            somaAcimaDiagonalSecundaria += matriz[i][j];
        }
    }
}
cout << "Soma dos itens acima da diagonal secundaria: " <<
somaAcimaDiagonalSecundaria << endl;

// Cálculo da soma dos itens abaixo da diagonal secundária
// Abaixo da Diagonal Secundária: Soma os elementos onde a linha é maior que N - 1 - j
(i > N - 1 - j).
int somaAbaixoDiagonalSecundaria = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        if (i > N - 1 - j) { // Elementos abaixo da diagonal secundária
            somaAbaixoDiagonalSecundaria += matriz[i][j];
        }
    }
}
cout << "Soma dos itens abaixo da diagonal secundaria: " <<
somaAbaixoDiagonalSecundaria << endl;

return 0;
}
*/
/* -----*/

```

```

/* ----- */
//JOGO DA VELHA
/*
int main () {
    char tabuleiro[3][3] = {{ '1', '2', '3'},
                           { '4', '5', '6'},
                           { '7', '8', '9'} };
    int jogador = 1;
    int escolha;
    char marca;
    bool ativo = true;

    while (ativo) {
        cout << "\nTabuleiro:\n";
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                cout << tabuleiro[i][j] << " ";
            }
            cout << endl;
        }

        //trocar os jogadores
        marca = (jogador == 1) ? 'X' : 'O';

        cout << "Jogador " << jogador << ", escolha um numero: ";
        cin >> escolha;

        switch (escolha) {
            case 1: if (tabuleiro[0][0] == '1') tabuleiro[0][0] = marca; else cout << "Posição já ocupada!\n"; break;
            case 2: if (tabuleiro[0][1] == '2') tabuleiro[0][1] = marca; else cout << "Posição já ocupada!\n"; break;
            case 3: if (tabuleiro[0][2] == '3') tabuleiro[0][2] = marca; else cout << "Posição já ocupada!\n"; break;
            case 4: if (tabuleiro[1][0] == '4') tabuleiro[1][0] = marca; else cout << "Posição já ocupada!\n"; break;
            case 5: if (tabuleiro[1][1] == '5') tabuleiro[1][1] = marca; else cout << "Posição já ocupada!\n"; break;
            case 6: if (tabuleiro[1][2] == '6') tabuleiro[1][2] = marca; else cout << "Posição já ocupada!\n"; break;
            case 7: if (tabuleiro[2][0] == '7') tabuleiro[2][0] = marca; else cout << "Posição já ocupada!\n"; break;
            case 8: if (tabuleiro[2][1] == '8') tabuleiro[2][1] = marca; else cout << "Posição já ocupada!\n"; break;

```

```

        case 9: if (tabuleiro[2][2] == '9') tabuleiro[2][2] = marca; else cout << "Posição já
ocupada!\n"; break;
        default:
            cout << "Escolha inválida! Tente novamente.\n";
            continue;
    }

    for (int i = 0; i < 3; i++) {
        if ((tabuleiro[i][0] == marca && tabuleiro[i][1] == marca && tabuleiro[i][2] == marca)
||
        (tabuleiro[0][i] == marca && tabuleiro[1][i] == marca && tabuleiro[2][i] ==
marca)) {
            cout << "Jogador " << jogador << " venceu!\n";
            ativo = false;
        }
    }

    if ((tabuleiro[0][0] == marca && tabuleiro[1][1] == marca && tabuleiro[2][2] ==
marca) ||
        (tabuleiro[0][2] == marca && tabuleiro[1][1] == marca && tabuleiro[2][0] ==
marca)) {
        cout << "Jogador " << jogador << " venceu!\n";
        ativo = false;
    }

    // Verificando empate
    bool empate = true;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (tabuleiro[i][j] != 'X' && tabuleiro[i][j] != 'O') {
                empate = false;
            }
        }
    }
    if (empate) {
        cout << "Velha!!\n";
        ativo = false;
    }
    jogador = (jogador == 1) ? 2 : 1;
}
return 0;
}
*/
/* -----*/

/* -----*/
//FORCA

```

```

/*
int main () {
    string palavra = "";

    //oferta ao usuario de escolher a palavra "multiplayer":
    cout << "Escolha a palavra (tudo minusculo sem acento!): ";
    // guardar toda a linha nao tem como por espaço!
    //      getline(cin, palavra);
    cin >> palavra;

    string letrastentativa = "";
    string palavraoculta(palavra.size(), '_');
    int tentativas = 8;

    while (tentativas > 0 && palavraoculta!=palavra) {
        cout << "\nPalavra: " << palavraoculta << endl;
        cout << "Tentativas restantes: " << tentativas << endl;
        cout << "Letras tentadas: " << letrastentativa << endl;

        char letra;
        cout << "Digite uma letra: ";
        cin >> letra;

        // Verifica se a letra já foi tentada
        if (letrastentativa.find(letra) != string::npos) {
            cout << "Você já tentou essa letra. Tente outra.\n";
            continue;
        }

        letrastentativa += letra; // Adiciona a letra tentada

        // Verifica se a letra está na palavra
        bool letraEncontrada = false;
        for (int i = 0; i < palavra.size(); i++) {
            if (palavra[i] == letra) {
                palavraoculta[i] = letra; // Revela a letra na palavra oculta
                letraEncontrada = true;
            }
        }

        if (!letraEncontrada) {
            tentativas--; // Reduz o número de tentativas se a letra não estiver na palavra
            cout << "Letra não encontrada!\n";
        }

        if (palavraoculta == palavra) {

```

```
        cout << "\nParabéns! Você adivinhou a palavra: " << palavra << endl;
    } else {
        cout << "\nVocê perdeu! A palavra era: " << palavra << endl;
    }
    return 0;
}
*/
/* -----*/
```